

HARDWARE BASED IMAGE CLASSIFICATION

Submitted by

NEHRU H [Reg No: RA1611004010118]
SHRIHARI C [Reg No: RA1611004010306]
JAYAPRABU S [Reg No: RA1611004010090]

Under the guidance of
Dr. S. MALARVIZHI, Ph.D.
(Professor, Department of Electronics & Communication
Engineering)

In partial fulfillment for the award of the degree
Of

BACHELOR OF TECHNOLOGY
In

ELECTRONICS AND COMMUNICATION
ENGINEERING



Of
FACULTY OF ENGINEERING AND TECHNOLOGY
S.R.M. Nagar, Kattankulathur, Kancheepuram District
MAY 2020

BONAFIDE CERTIFICATE

Certificate that this project report titled “Hardware based Image Classification” is the bonafide work of “**C.SHRIHARI [RA1611004010306], NEHRU.H [RA1611004010118], JAYAPRABU [RA1611004010090]**”, who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

DR.S.MALARVIZHI

Professor

Dept. of ECE

Signature of the Internal Examiner

ACKNOWLEDGEMENT

We'd like to take this humble opportunity to thank our Chancellor Dr. T. R. Paarivendhar for allowing us this opportunity on such a great platform at the completion of this course. We'd want to convey our deepest thanks to our Vice Chancellor Dr. Sandeep Sancheti for their constant encouragement and understanding. We also would like to express our sincere thanks to our Director Dr. C. Muthamizhchelvan for the constant support and guidance from his side. We would like to hand our gratitude to the Head of Department Dr.T. Rama Rao for the constant insight and support that he has provided over this project completion. We the members of this project would like to express our deepest appreciations to our guide, Dr. S. Malarvizhi, for her valuable guidance and counseling, constant encouragement, personal care, timely help and providing us with an excellent atmosphere for doing research. All through the work, in spite of her busy schedule, she has extended her cheerful and cordial support to us for completing this research work. We would also like to extend our thanks to Dr. J. Manjula, Dr. Rajesh Agarwal and Mr. S. Yuvaraj for their critical reviews and suggestions allowing us to go a step beyond our limits at the critical stages of our project to shape it into its final product.

C.SHRIHARI (RA1611004010306)

NEHRU.H (RA1611004010118)

JAYA PRABU (RA1611004010090)

ABSTRACT

Tasks can be executed with more efficiency on FPGA's due to their vast parallel processing capability. This is because their hardware can be reprogrammed using software to suit the needs of the application. One task which requires high parallel processing is Deep learning. It requires large number of simple tasks to be performed in a short period of time making them a good model to demonstrate the potential of FPGA's. In this project we aim to implement one such deep learning algorithm - a Convolutional Neural Network (Lenet-5) to classify hand written digits of MNIST data set using a PYNQ Z2 FPGA using Xilinx workflow which includes Vivado and Vivado HLS tools along with Jupyter Notebook to achieve required accuracy.

TABLE OF CONTENTS

	Page
LIST OF FIGURES	3
LIST OF TABLES	3
 1. INTRODUCTION	 4
2. LITERATURE SURVEY	5
2.1AN EFFICIENT CNN ARCHITECTURE FOR IMAGE CLASSIFICATION	5
2.2ACCELERATION OF IMAGE CLASSIFICATION WITH CAFFE FRAMEWORK USING FPGA	5
2.3FPGA BASED CNN FOR HYPERSPECTRAL IMAGE CLASSIFICATION	6
2.4DEEP LEARNING (ADAPTIVE COMPUTATION AND MACHINE LEARNING)	6
3. SYSTEM DESIGN AND DESCRIPTION	7
3.1PROBLEM STATEMENT AND OBJECTIVES	
3.2DESIGN FLOW	7
3.3CNN ARCHITECTURE	12
3.4IP BLOCKS	16
3.5CODING AND ACCURACY	20
3.6UTILIZATION	25
4. STANDARDS INVOLVED AND MULTI- - DISCIPLINARY ASPECTS	27
5. LIMITATIONS OF FPGA AND MEMBERS CONTRIBUTIONS	29
6. REALISTIC CONSTRAINTS	30
7. CONCLUSION AND FUTURE SCOPE OF THE PROJECT	31
ENGINEERING STANDARDS	32
REFERENCES	33

LIST OF FIGURES

	Page
a) IP Block Design Flow	8
b) Application design	9
c) Hardware design flow	11
d) Pooling layer calculation matrix	13
e) CNN Architecture layers	15
f) Convolutional layer design	16
g) Demonstration of Convolution process in FPGA	17
h) Maxpooling IP design	18
i) Fully Connected layer IP design	18
j) IP Blocks chained to form Neural Network	19
k) LeNet-5 design	19
l) Accuracy results	21
m) Confusion matrix output	22
n) Sample MNIST dataset	23
o) Sample image of actual and predicted data	24
p) Post Synthesis Utilization chart	26
q) Post Implementation chart	26

LIST OF TABLES

1. Technical parameters specifications of various layers	15
2. Resource allocated components for Utilization charts	26

LIST OF ABBREVIATIONS

1. SCIG: Synchronous Convolution Input Graph block
2. SMM: Synchronous Matrix Multiplication
3. CNN: Convolutional Neural Network
4. LeNet: A basic neural network architecture proposed by Yann LeCun
5. FPGA: Field Programmable Gate Array
6. IDE: Integrated Development Environment
7. DNN: Deep Neural Network
8. FC: Fully Connected
9. HLS: High Level Synthesis
10. RTL: Register Transfer Logic
11. VLSI: Very Large Scale Integration
12. VHDL: Very High Speed Integrated Circuit Hardware Description Language
13. FC: Fully Connected layer
14. GPU: Graphics Processing Unit
15. ReLU: Rectified Linear Unit layer
16. IP: Intellectual Property blocks
17. MNIST: Modified National Institute of Standards and Technology database

CHAPTER 1

INTRODUCTION

The project is divided into a number of parts starting with choosing the hardware that is compatible with our purpose, user friendly and cheap. The next step is to choose a suitable image classification algorithm and data set that is compatible with it. The design flow process is then followed to obtain a bit stream file which is used to operate the FPGA.

A suitable IDE is chosen and the coding required to implement the algorithm is written. The coding includes a way to visually see the results of the algorithm. The data set is processed by the FPGA and the results are obtained, this also includes the utilization results.

The FPGA used is PYNQ Z2 which is manufactured by TUL for Xilinx Zynq platform with special support for PYNQ which gives it support for python programming language. The software used for configuring the FPGA is Xilinx Vivado and HLS tools.

CHAPTER 2

LITERATURE SURVEY

2.1 AN EFFICIENT CNN ARCHITECTURE FOR IMAGE CLASSIFICATION:

- Targeted board: Artix-7 series FPGA
- The developed architecture attained a significant improvement in speed compared to existing architecture working at 300MHz maximum operating frequency.
- The research implements a 3 layered CNN architecture comprising of convolutional, mean pooling and fully connected layers respectively.
- They do not have any graphical user interface for accessing images from the dataset.

2.2 ACCELERATION OF IMAGE CLASSIFICATION WITH CAFFE FRAMEWORK USING FPGA:

- A modified version of Caffe to effortlessly port it into the ARM (Zynq 7000 based) processor of FPGA.
- A hardware accelerator designed on Xilinx SDSoC Environment that leverages the FPGA architecture to perform the image classification algorithm.
- They have used a high end board which has practical purposes for industry applications only.
- This is a basic implementation of DNN using adequate modifications in caffe dependencies for the FPGA; however caffe is already available on Zynq compatible Xilinx boards.

2.3 FPGA BASED CNN FOR HYPERSPECTRAL IMAGE CLASSIFICATION:

- A hardware model for FPGA based implementation of CNN is designed by hardware description language to classify hyper-spectral images.
- Target board: Any High end FPGA
- They have used a 3 layer CNN comprising of convolutional, batch normalization and dense layers respectively.
- It is only a model for determining the ease of versatility of FPGA's as compared to GPU's which poorly perform for hyper-spectral classification tasks.

2.4 DEEP LEARNING (ADAPTIVE COMPUTATION AND MACHINE LEARNING):

- MIT Deep Learning Book
- A free comprehensive book for knowledge on Deep Learning methodologies.
- This book provided the detail understanding of Neural networking terminologies, insight into various models used in Deep learning and numerical computations for various layers.

CHAPTER 3: SYSTEM DESIGN AND DESCRIPTION

3.1: PROBLEM STATEMENT AND OBJECTIVES

Problem Statement:

Creating automated image classification systems are in dire need for various applications, Machine learning and Artificial Intelligence based approaches are easier to model in GPU's however the quality of GPU's also matter in its application, however building systems as equivalent to GPU's require higher complexities, thus building of neural network based application to run on reconfigurable hardware would be our vision throughout this project.

Objectives:

- To implement an efficient Convolutional Neural Network on FPGA for improved utilization parameters.
- To classify images from dataset (MNIST) via Pre-trained Neural Networking model (CNN).
- To obtain a real time system classifying images in sets of images.
- Creating a model of classification which in future can be used for variety of Neural Network based systems.

3.2: DESIGN FLOW

This section describes all important and essential processes required to build VLSI subsystems as well as integration for Neural Networks and its Databases.

VLSI subsystems required in the project are created by using programming languages like Verilog and VHDL, however the main purpose of these codes are to create interconnection between various combinational and sequential circuits which are to be programmed on a configurable hardware, Such a system allows us to create complicated design for prototyping and testing projects in numerous fields of latest generation applications. Our VLSI systems are designed to hold CNN applications.

This project has two separate design methodologies, they are:

- Creating VLSI logic circuit design
- Creating Application Interface

VLSI Designing:

This design involves in designing subsystems, integrating them, validating designs and finally troubleshooting the design for no errors in the system.

The subsystems as mentioned above are to be coded using Verilog language, the subsystems designed are SCIG, SMM, Maxpool, Register Slices and FC blocks, the functions of these blocks will be further explained in succeeding chapters. Prior to designing the Verilog code, we must create a C code describing our system which would be later converted into Verilog language, this is done because Verilog systems like ours can be too long and thus time consuming for designing from scratch. Thus we make use of Vivado HLS software for performing this procedure, however not all designs need to be approached in such a manner.

If the Verilog codes are obtained successfully, the designs are further synthesized, simulated and RTL simulated, which are key procedures in

all Verilog Design Flows. The resulting design is called an IP, thus each VLSI subsystem is termed as an IP, various IP's are interconnected and appropriate signals, clocks and intermediary interconnection wires are connected to form the VLSI System. The design flow is shown in below Figure-a.

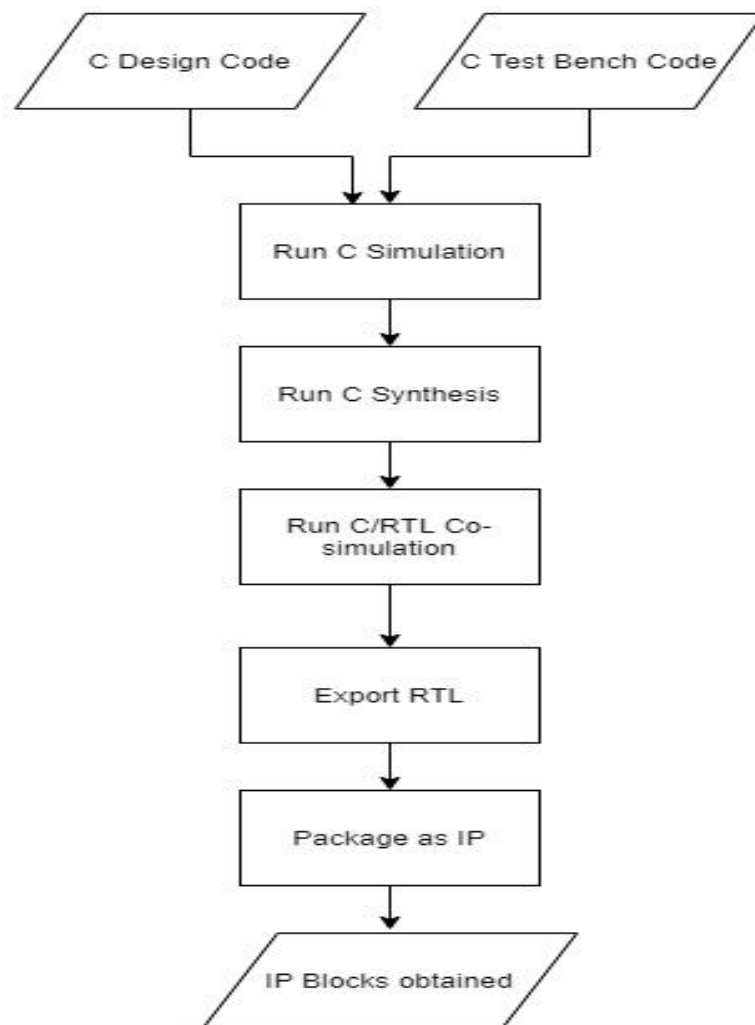
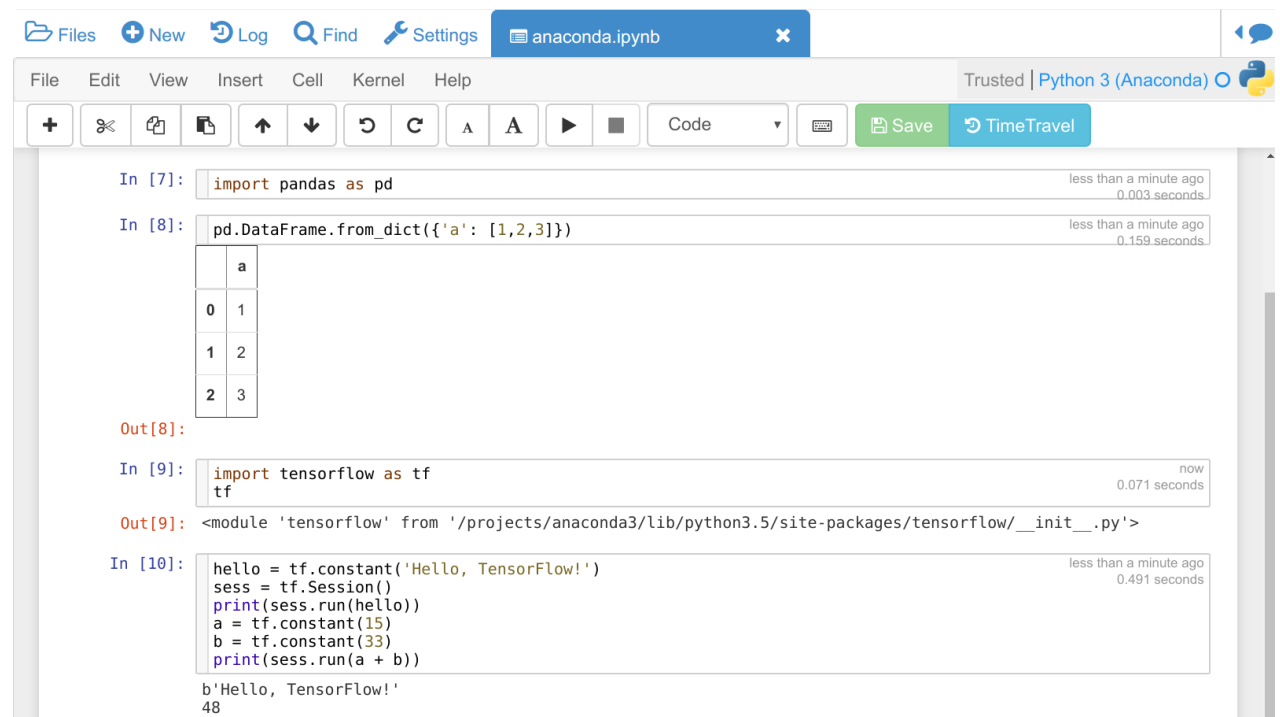


Figure-a: IP Block design flow

Application Design:

The Application Interface for the project involves in creating a Convolutional Neural Network model for training and testing images for Image Classification using Python Programming Language. The sole purpose of such a methodology is to link this Neural Network onto the configurable logic design for parallel processing capabilities.

The Application is basically a CNN code optimized for five layers; this network is also named as LeNet-5. This Interface also provides us to write and debug code, the software used is Jupyter notebooks which are web based API, used to create and share codes. The Jupyter interface is as shown below in Figure-b.



The screenshot displays a Jupyter Notebook window titled 'anaconda.ipynb'. The interface includes a top toolbar with icons for file operations, a menu bar (File, Edit, View, Insert, Cell, Kernel, Help), and a status bar indicating 'Trusted | Python 3 (Anaconda)'. The notebook contains four input cells and their corresponding outputs:

- In [7]:** `import pandas as pd` (Execution time: less than a minute ago, 0.003 seconds)
- In [8]:** `pd.DataFrame.from_dict({'a': [1,2,3]})` (Execution time: less than a minute ago, 0.159 seconds). The output is a DataFrame with a single column 'a' containing values 1, 2, and 3.

	a
0	1
1	2
2	3
- Out[8]:** (Empty output)
- In [9]:** `import tensorflow as tf` (Execution time: now, 0.071 seconds)
- Out[9]:** `<module 'tensorflow' from '/projects/anaconda3/lib/python3.5/site-packages/tensorflow/__init__.py'>`
- In [10]:** `hello = tf.constant('Hello, TensorFlow!')
sess = tf.Session()
print(sess.run(hello))
a = tf.constant(15)
b = tf.constant(33)
print(sess.run(a + b))` (Execution time: less than a minute ago, 0.491 seconds). The output is:
b'Hello, TensorFlow!'
48

Figure-b: Application design

Hardware Designing

As explained in the previous section the VLSI Subsystem is also known as IP Blocks, in this section you will understand the definition, function and importance of IP's in a Hardware designing part.

Intellectual Property (IP) as in general terms relate to an idea or a solution which is unique to a person and he is the sole owner for the product, which complies that if anyone would want to access another idea they would have to purchase the product for accessing, similarly in Xilinx it refers to pre-configured logic functions that can be used in your design. Xilinx Vivado provides a wide selection of IP's which are optimized for Xilinx FPGAs. They can include functions delivered through the Xilinx software, through the Xilinx Architecture Wizard, as standalone archives, from third parties, through Xilinx Platform Studio (XPS), or through System Generator.

However to support our LeNet-5 model which is derived from CNN, we had to create our own custom made IP for the project. All the IP's are created from the same design flow procedure as explained in the previous section, thus we shall shed light into the various IP Blocks we chained for our project.

Once the subsystems for SCIG, SMM, Maxpool, Register Slices and FC blocks are created they are connected accordingly and the design is validated for connection errors and improper clock synchronizations. Then the discrete design consisting of many blocks is compiled into a single block design and it is termed as HDL Wrapper. The HDL Wrapper also gives information of the final Verilog code used for the design, later on the design is synthesized which is converting the Verilog code to gate level representation. Implementation causes the gate level representation to be reconfigured into the FPGA. Finally this design is saved as Bit stream file which is the programmed design or the project. This bit stream is linked accordingly with the application interface for parallel processing task. This design flow is as represented in the below Figure-c.

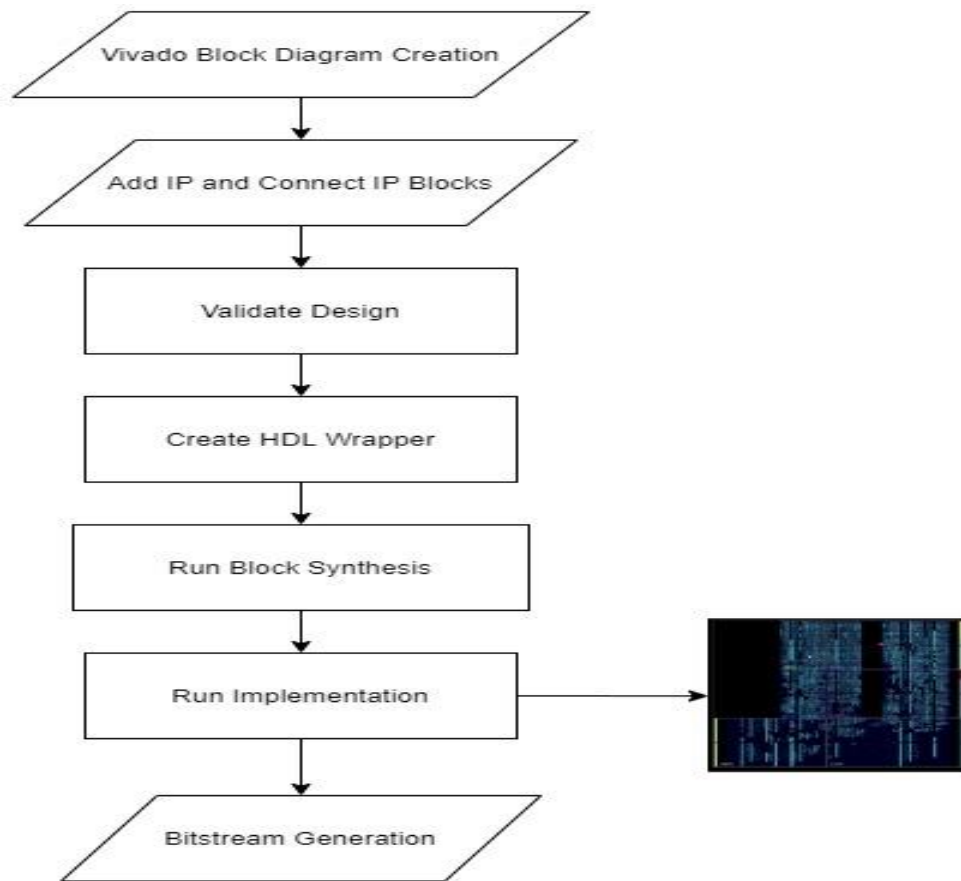


Figure-c: Hardware design flow

3.3: CNN ARCHITECTURE

Convolutional Neural Network (CNN):

A Neural Network can be represented by hardware and/or software shaped after the operation of neurons in the human brain. CNN's have their "neurons" arranged like the frontal lobe; it is responsible for visual stimulation in humans so that pixels can be interpreted in a similar fashion. A CNN consists of an input layer, one or many hidden layers and an output layer that includes multiple pooling layers, fully connected layers, convolutional layers and normalization layers.

Convolution Layer:

Convolutional Layer is one of the first layers of the CNN. It is the part where processing happens and is very integral part of the CNN. The layer consists of parameters which are a number of filters or kernels, which have a small field of reception but will extend to the entire depth of the input volume. Each filter is convolved across the width and height of the part of the input image it is responsible from the input image, it computes the dot product between the output of the filter and the input and it produces a 2-D activation map of that filter. The network uses filters that activate when it detects a specific feature at some spatial position in the input

Pooling Layer:

Pooling is an important concept in CNN's, it is a down-sampling performed in a non-linear way. Out of various non-linear functions that can be implemented to perform pooling max pooling is the most common. The input image is partitioned into a bunch of rectangles and the maximum of each sub-region is given as the output. Below Figure-d shows calculation of the pooling layer.

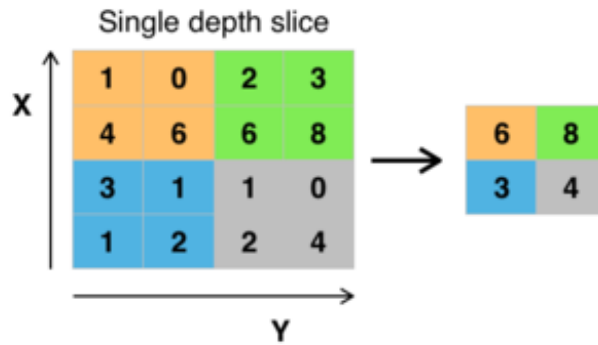


Figure-d: Pooling layer calculation matrix

Fully Connected Layer:

Fully Connected layers in Neural Network's are the ones where all the inputs from one layer are connected to an activation unit of the next layer. Usually the last few layers are fully connected layers which take the data extracted by previous layers and compile them to form the final output.

LeNet-5 Architecture:

Convolution Layer 1:

The first layer of the CNN is a convolution layer and it acts as the input layer for the algorithm.

It takes 28*28 Grey scale images as input and converts them into matrices using feature maps. For every image 20 feature maps are used with dimension 5*5 and this reduces the dimension of the matrix from 28*28 to 24*24.

Pooling Layer 1:

The first pooling layer uses a max pool function to down-sample the matrices from 24*24 to 12*12 using a filter with size 2*2. This reduces number of calculations needed by the next convolution layer by half.

Convolution Layer 2:

The next convolution layer is similar to the first one and uses the sliding window to further generate 50 feature maps with filter of dimension 5×5 and in the process converts the matrices to size 8×8 .

Pooling Layer 2:

The last pooling layer down-samples the matrices from 8×8 to 4×4 using the max pool function and filter size 2×2 .

First Fully Connected Layer :

The first FC layer converts the feature maps of size 4×4 to 500 units of size 1×1 .

Second Fully Connected Layer:

The final layer converts the 500 units to 10 and these 10 correspond to the 10 classes of the image dataset and the 10 outputs possible from the algorithm.

Output Layer:

There is a Fully Connected softmax output layer which gives us one of the 10 possible values corresponding to the digits from 0 to 9. Below figure-e demonstrates the CNN architecture used for our project, Table-1 displays each layer's technical parameter specifications.

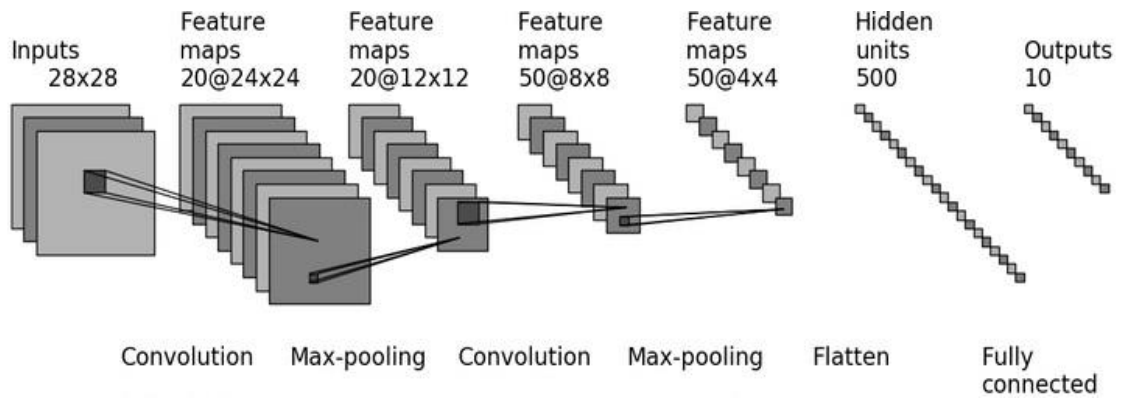


Figure-e: CNN Architecture layers

INPUT	IMAGE	FEATURE MAP	INPUT SIZE	KERNAL SIZE	ACTIVATION
1	CONVOLUTION	20	28*28	5*5	ReLU
2	POOLING	20	24*24	2*2	Max Pool
3	CONVOLUTION	50	12*12	5*5	ReLU
4	POOLING	50	8*8	2*2	Max Pool
5	FULLY CONNECTED	-	4*4	5*5	ReLU
6	FULLY CONNECTED	-	1*1	1*1	ReLU
7	OUTPUT	-	1*1	-	SOFTMAX

Table-1: Technical parameters specifications of various layers

3.4: IP BLOCKS

Three CNN layers, namely convolution layer, pooling layer, and fully-connected layer, have been constructed using Vivado HLS. ReLu layer was included in each of the three layers, as each layer IP has the option of whether to directly rectify each output data as the output feature map streams out.

Convolutional Layer:

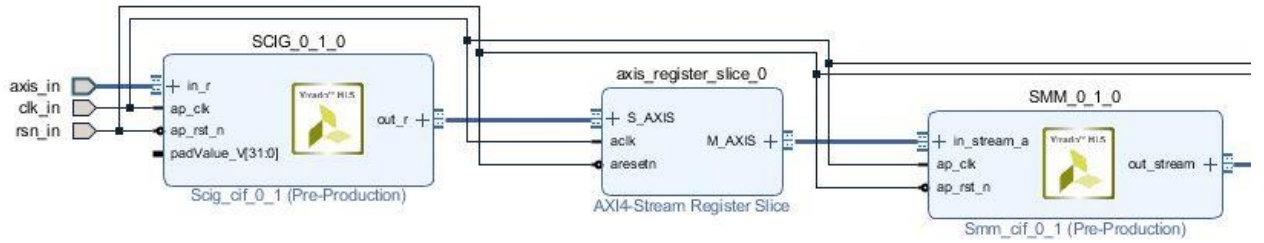


Figure-f: Convolutional layer design

The above image (Figure-f) shows the section of Convolutional operation in the LeNet-5 Hardware design. The convolution between the input feature map X and the kernel W can be achieved by expanding the feature map into a large 2D matrix, M , followed by matrix multiplication between M and unrolled 2D version of M .

The SCIG IP expands the input feature map into column vectors, where each column vector contains the patch of input feature map which will perform dot-product with kernel. Although this operation essentially expands the memory usage of input feature map by replicating data, this operation has effectively transformed convolution into matrix multiplication.

With AXI4-Streaming interface, "SCIG" memory footprint problem can be effectively eliminated. The outputs of "SCIG" block can immediately

stream out to the subsequent block; hence there is no need for buffering the output feature map.

The second stage of convolution layer is matrix multiplication. Xilinx Vivado HLS has extensive support for implementing AXI4-Streaming matrix multiplications on Zynq FPGA. As shown in Figure-g, in matrix multiplication $A*B=C$, each value in C is generated by dot-product between a row of 'A' and a column of 'B'. This dot-product can be implemented in parallel on FPGA architecture, and pipelined so that one output value can be generated per clock cycle.

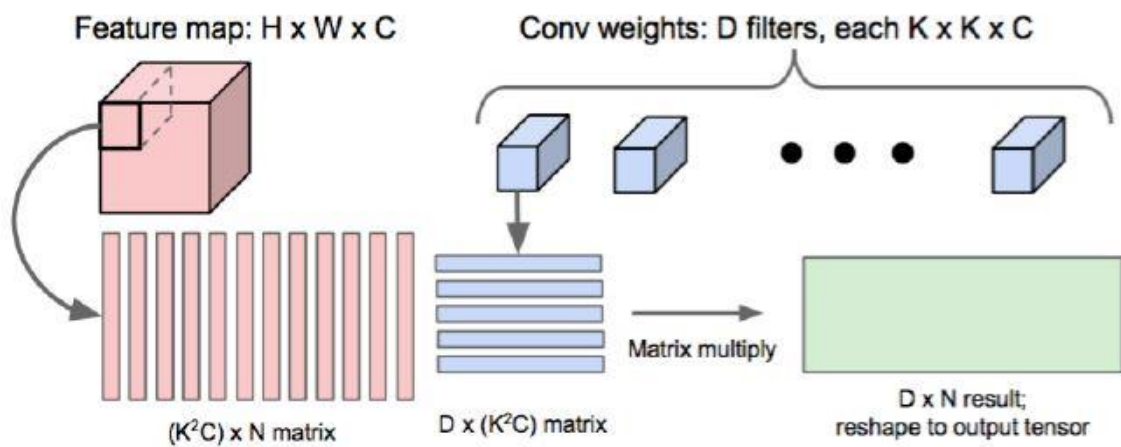


Figure-g: Demonstration of Convolution process in FPGA

Pooling Layer:

Pooling layer performs down-sampling on input feature map, by outputting maximum or average of sub graphs. In AXI4-Streaming interface, pooling can be performed in similar concepts to SCIG unit. It gradually reduces the spatial dimensions of the representation to bring down the number of parameters and computations needed. It operates independently on each feature map. The maxpooling IP block is as shown below in Figure-h.

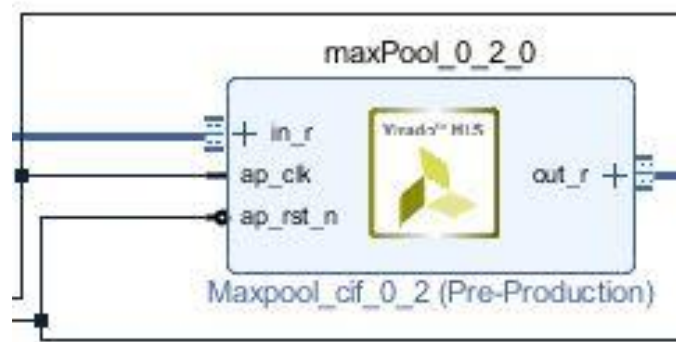


Figure-h: Maxpooling IP Design

Fully Connected Layer:

Fully-connected layer performs dot-product between input feature map and weights. Hence, in the off-line, a fully-connected layer can be easily converted into convolution layer. The Figure-i shows Fully Connected Layer IP.

For example, consider an input feature map of dimension (512x7x7), which will go through a fully connected layer with 4096 outputs. The weights of this fully-connected layer, which have dimension (4096x25088), can be reshaped into (4096x512x7x7), forming a convolution layer with kernel dimension = 7, input channel = 512, stride = 1, and zero-padding = 0.



Figure-i: Fully Connected Layer IP design

However some of the layers are in repetition for making the actual LeNet-5 design, below Figure-j and Figure-k shows the actual design.

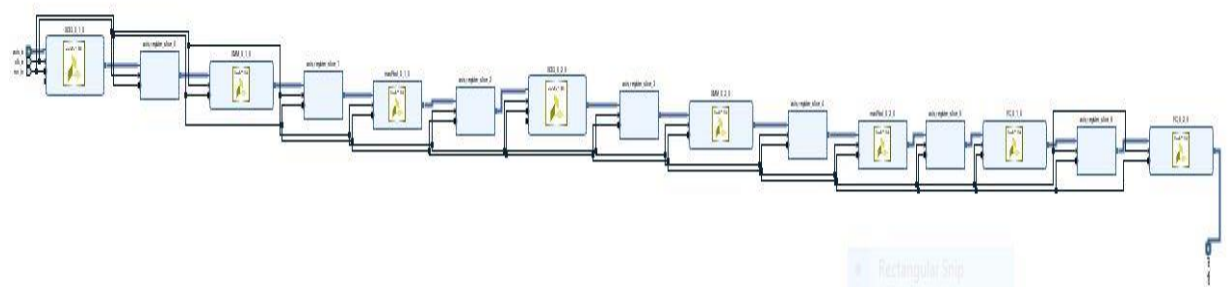


Figure-j: IP Blocks chained to form our Neural Network

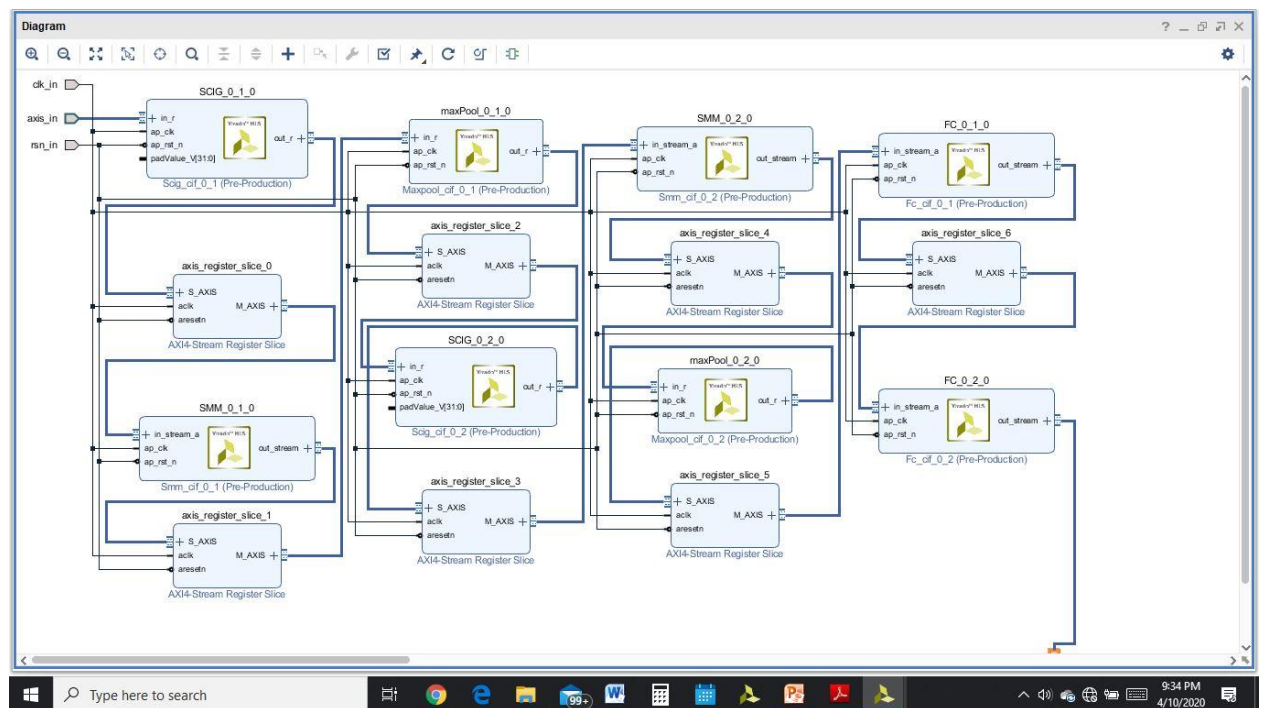


Figure-k: Displays the LeNet-5 design in a single window

3.5: CODING AND ACCURACY

Accuracy:

The software used to interact with the FPGA board is Jupyter notebook which is an open source web application which takes python code as input.

Even though the CNN is programmed into the FPGA using the bit stream file generated by the Verilog code in Vivado it needs to be separately designed with the same parameters and architecture in the python script.

Since this is a pre-trained algorithm the weights need to be extracted and loaded onto the FPGA.

Caffe framework is used to design the CNN and Theano is used for compatibility reasons.

After the CNN is designed and weights loaded it is now ready to classify images. The dataset is loaded from the SD card and prepared to be sent in series to the BRAM. Before that the images are reshaped to 28*28 format and the labels of the data used for testing are obtained. The data is split and 500 images are used for testing the accuracy of the algorithm. Figure-1 displays accuracy obtained from Jupyter notebook.

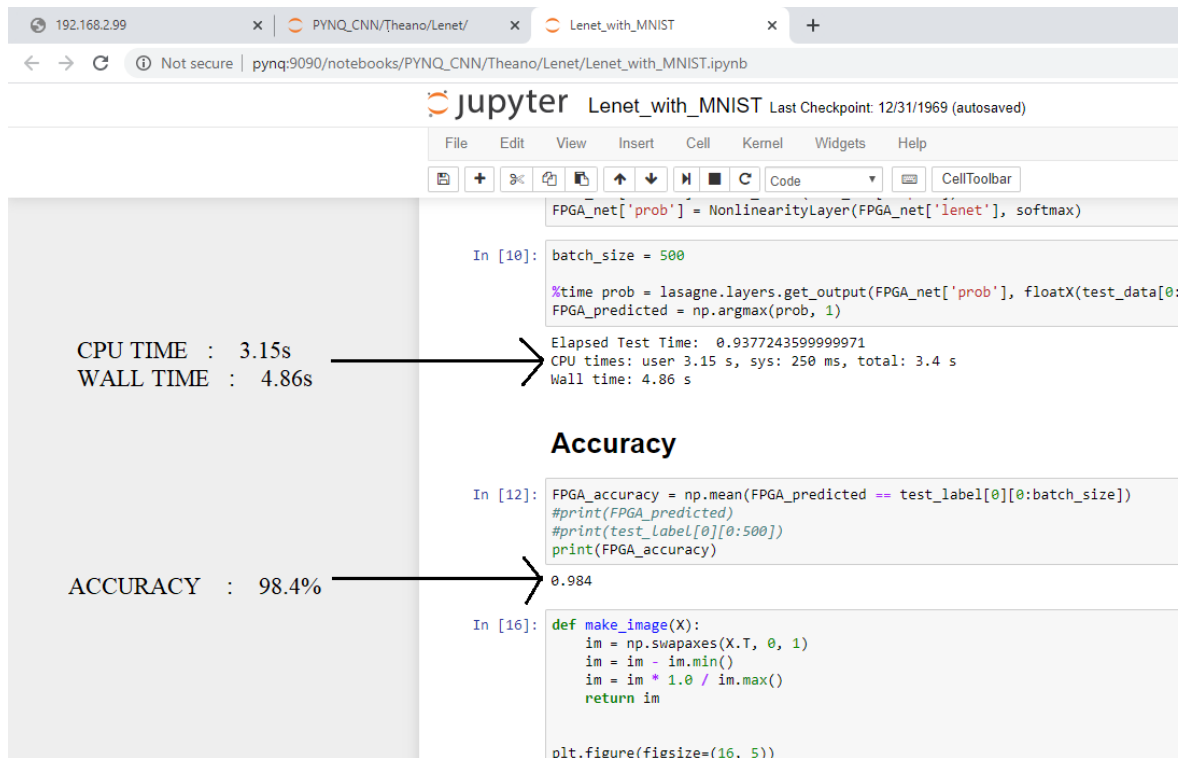


Figure-l: Accuracy Results

As we can see from the above screenshot the obtained accuracy is satisfactory.

In the first section of the screenshot the algorithm is run and the images are classified one by one and the process takes 4.8s seconds on the used hardware. Here the algorithm classifies every image and makes a prediction which is stored. The number of correct predictions is stored by comparing the predicted values with the actual values.

As we can see from the picture the accuracy of the algorithm is obtained by dividing the number of correct predictions by the batch size and the output is 0.984 or 98.4%.

The CNN needs to be trained with the data set being used so that good accuracy can be achieved

The dataset is taken and 500 images are reserved for testing the accuracy and the rest of it is used to train the CNN and generate the weights required. We trained the CNN using Google Colab which is a cloud server allowing for fast computations. 10,000 images were used to train

the algorithm with 20 epochs. An epoch is when all the images have been used to train the algorithm once. Therefore with 20 epochs the CNN was trained with these 10,000 images 20 times. This ensured that the accuracy would be really good.

Confusion Matrix:

The results can further be visualized better using a confusion matrix which is a typical method used to understand the efficiency of a CNN as it gives us the information regarding which image was misclassified. It helps us understand the shortcomings of the CNN and tells us which images it is most likely to misclassify. The confusion matrix for our CNN is shown as below in Figure-m.

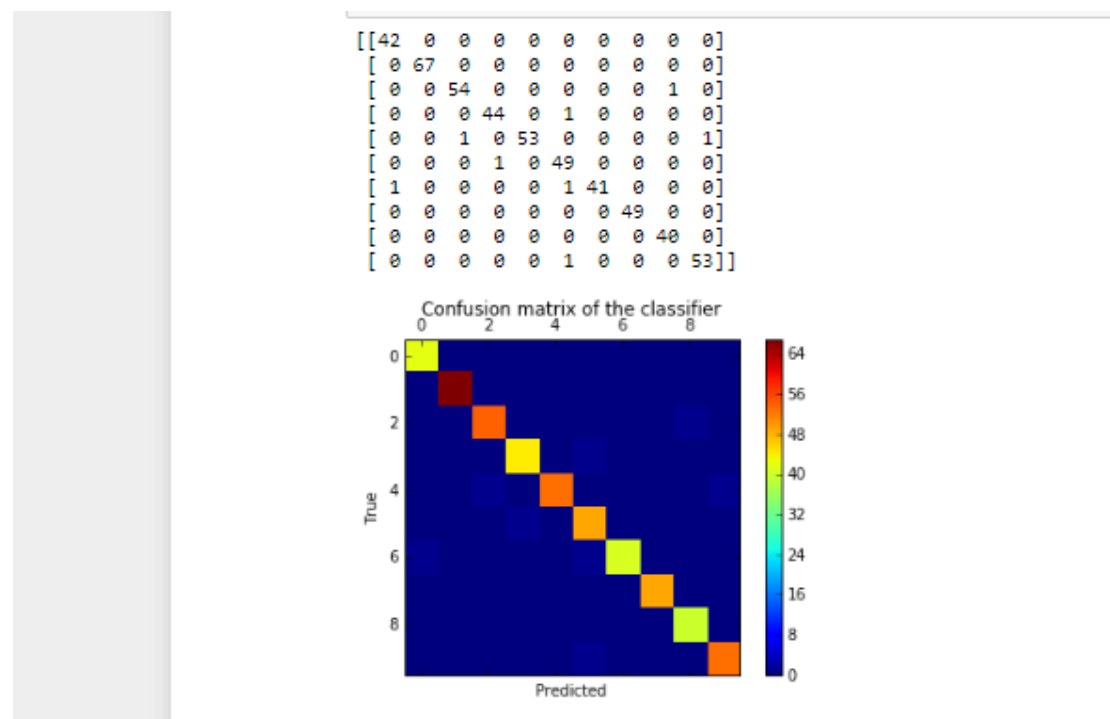


Figure-m: Confusion matrix output

Application:

Even though the algorithm is capable of classifying images from a number of datasets we are limited by the resolution of images that can be given as input. Since this particular CNN requires images of size 28*28,

it is a tough task to find images of such low resolution while still preserving enough detail to hold enough information to classify. It must also be taken into account that the accuracy of the CNN is dependent on the number of images it is trained with and hence we must make sure the dataset contains at least a few thousand images. This leaves us with a small number of existing datasets to choose from, out of these MNIST dataset (Figure-n) was chosen due to images being greyscale which and in particular only contains black and white elements making it simple and quick to process.

Image resolution : 28*28

Number of images : 500

Number of classes : 10

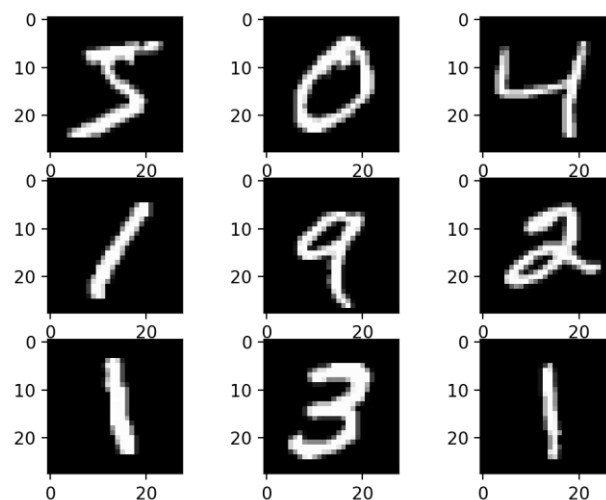


Figure-n: Sample MNIST dataset

Since Jupyter Notebook contains the python script editor being used all the libraries supported by its PC version is supported here, the Caffe dependencies and all the other required libraries can be stored on the SD card and can be called when needed. One such library used is Matplotlib which is used to plot the image with the actual value at the top left corner and the predicted value on the bottom left corner.

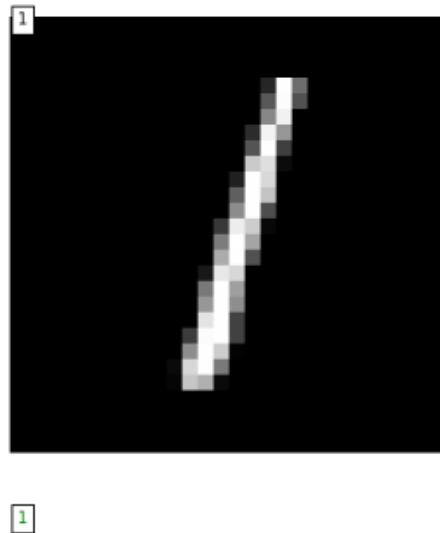


Figure-o: Top left corner displays actual data while bottom left corner shows predicted data

Another such library used is called scikit learn which is used to plot the confusion matrix and the coloured representation of it below. NumPy is used to do all mathematical calculations as it offers easy to use functions to shorten code and make it more readable.

3.6: UTILIZATION

The resources available on the chosen PYNQ Z2 board were sufficient to classify the images and hence it was chosen. Another reason is it supports Jupyter Notebook as its script editor and allows for deployment using python programming language which is widely used for machine learning applications.

The Hardware specifications of PYNQ Z2 board are as follows:

It consists of a 650 MHz dual core A9 processor with 512 Mb DDR3 RAM and 13,300 logic slices with 6-input LUT's and 106,400 Flip-Flops and 630 KB Block RAM with 220 DSP Slices.

- Logic Slices: They consist of 6 input LUT's and 8 Flip-Flops.
- LUT: Look Up Tables determines what the output is for any given inputs. FPGAs typically implement combinational logic with LUT's, and when it gets configured, it fills in the table output values, which are termed "LUT-Mask", and is physically composed of SRAM bits.
- BRAM: Block Random Access Memory (BRAM's) are used for storing large amounts of data inside of your FPGA.
- DSP Slices: They are the primary tool used by FPGA's to implement signal processing functions. One of the more common DSP operation used is Multiply-Accumulate.

After the synthesis and Implementation of the block diagram in Vivado the resources utilized is generated allowing us to know if the chosen hardware is appropriate.

The utilization outputs were generated as follows:

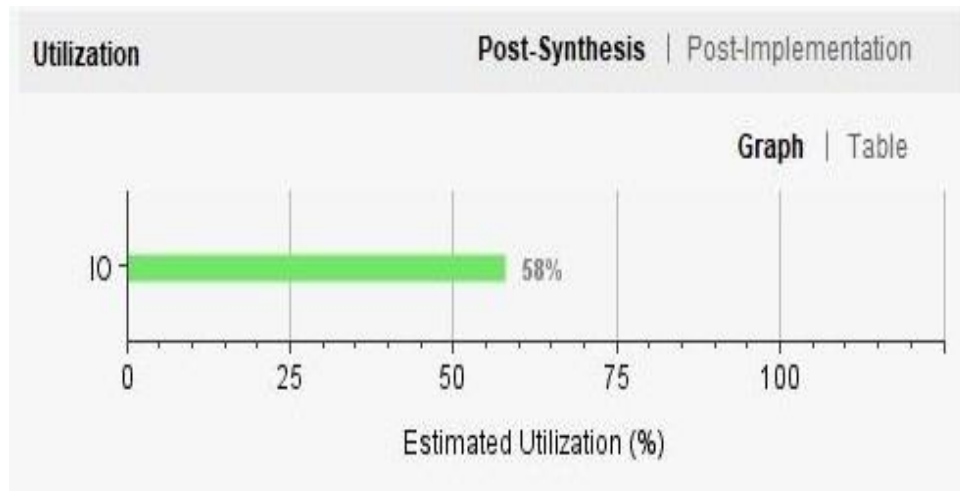


Figure-p: Post Synthesis Utilization chart

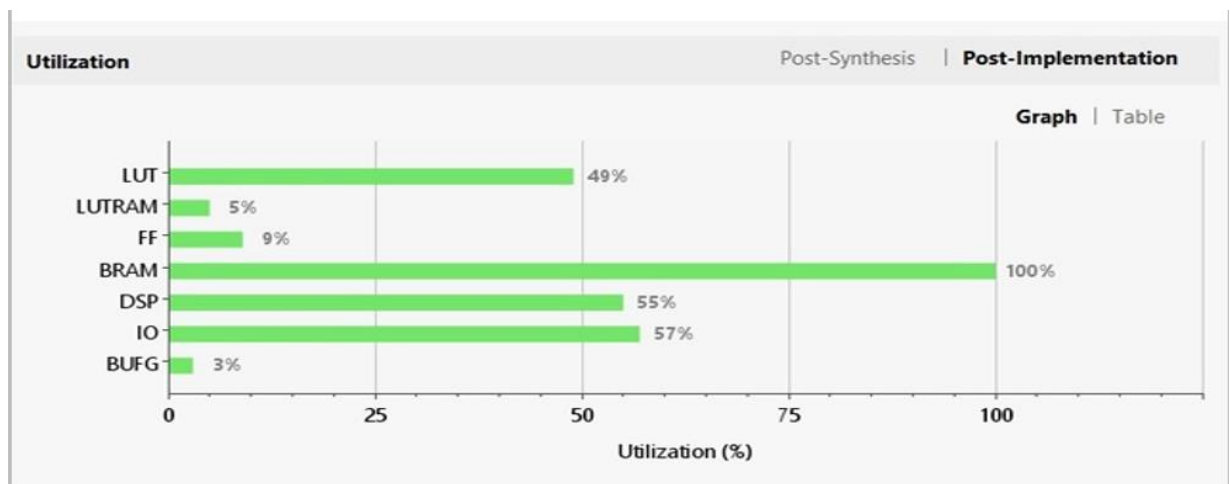


Figure-q: Post Implementation utilization parameters chart

UTILIZED ELEMENTS	LUT'S	FLIP FLOP'S	BLOCK RAM	DSP SLICES
RESOURCE ALLOCATION	26,068/53,200	95,760/1,06,400	630Kb	121/220

Table-2: Resource allocated components for Utilization charts

CHAPTER 4

STANDARDS INVOLVED AND MULTIDISCIPLINARY ASPECTS

STANDARDS INVOLVED

The standards used in this project come from the IEEE library. They are part of the VHDL packages which includes a number of packages which aid in FPGA programming. They can be called by first adding the line ‘Library IEEE;’ followed by ‘use + the package used; ’

In this particular project the packages used are

1) IEEE 1164 or IEEE Standard Logic - It is called using the line

‘use IEEE.std_logic_1164.all; ‘

and was established in 1993 and is still active. It defines the logic values to be used in design of FPGA’s, in VHDL hardware description language.

2) IEEE numeric std- It is called using the line

‘use IEEE.numeric_std.all; ‘

and is still active. It includes the numerical operators used as a part of VHDL like +, -, /, * along with mod, rem, < and >.

It supports signed and unsigned values and can give Boolean or numeric outputs.

MULTI-DISCIPLINARY ASPECTS

This project includes concepts from a number of disciplines with the primary being FPGA design using VHDL. The PYNQ Z2 FPGA we are using is programmed using the VHDL code generated by the Xilinx software. This is supported by a number of concepts from signal processing which includes Image processing which is achieved using VHDL. An image processing algorithm (CNN) is used to classify the images.

Knowledge of python programming language was used to showcase the output and control the FPGA. Basic computer skills were necessary to connect all the peripherals to the FPGA, computer and perform the tasks required.

MEMBERS CONTRIBUTIONS

- 1) SHRIHARI.C (RA1611004010306) - Initial Project Research
 - Assisted in Report Preparation
 - Obtained first simulation result

- 2) NEHRU.H (RA1611004010118) - Study and Review of Base paper
 - Assisted in Report Preparation
 - Obtained final simulation result

- 3) JAYA PRABU.S (RA1611004010090) - Preparation of Poster
 - Hardware acquirement
 - Creating Presentations

CHAPTER 5

LIMITATIONS OF FPGA

- 1) Cost - FPGA's are still more expensive as compared to conventional hardware alternatives like CPU's or GPU's.
- 2) Development difficulties - Making algorithms work on FPGA's is a lot more tedious and requires more prerequisite knowledge than implementing the same algorithm on other systems. Companies like Xilinx have their staff working on simplifying the process and make FPGA development accessible to more people especially those with less experience.
- 3) Lack of Resources - Since development with FPGA's is comparatively newer and less popular there is a lack of resources to learn their working and apply it. The situation is improving though as more people show interest in using FPGA for their needs.

CHAPTER 6

REALISTIC CONSTRAINTS

The realistic constraints are as follows:

- 1) Data set limitation - Since the chosen Image classification algorithm is simple and cannot support images with resolution more than 32×32 it limits the possible data sets to a small number.
- 2) Hardware limitation - The hardware used here is a PYNQ Z2 board which is a low-mid range FPGA more suitable for beginners in the field. This means it cannot support large algorithms which can in turn increase the possible data sets that can be used.
- 3) Peripherals Requirement - Other than the FPGA board a number of IO accessories are required to make the project functional. This includes a PC to train the CNN and send commands to the FPGA. An SD card is required to boot into the image file and is the only method of file transfer into the board.
- 4) Internet Dependancy - Since the project code is written in Jupyter Notebook which is an online python development platform an internet connection and Ethernet port is required to make it work.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE OF THE PROJECT

Conclusion:

Thus an Image Classification algorithm was implemented on a FPGA with a data set as input and the images were classified to one of the 10 classes. The accuracy obtained was satisfactory and the resource utilization of the hardware was obtained with a PYNQ Z2 board used as the hardware platform.

Future Scope:

This project can be improved upon in a number of ways be it software or hardware. The hardware chosen for the project was limited by the budget; hence with a higher price point to choose a better FPGA board more development opportunities can be unlocked. This increases the algorithms that can be implemented on the FPGA as a more complex CNN's with more layers require more hardware capacity to run without bottlenecks. If a more complex algorithm can be used it allows for an image of larger resolution to be used as input. Instead of using a 28*28 image as input we can use an image with resolution 227*227 which has a lot more detail and can therefore hold more information. One such more complex CNN is AlexNet which is a pre-trained algorithm and has shown excellent results.

ENGINEERING STANDARDS

The standards for the project undertaken are as follows:

Area	Codes & Standards
Economic	This project keeps costs into consideration as the analysis is restricted to affordable FPGA's.
Environmental	This project is not expected to entail Environmental constraints.
Ethical	This project is not expected to entail Ethical constraints.
Health and Safety	This project is not expected to entail Health and Safety constraints.
Manufacturability	FPGA's have shorter manufacturing time; also the design can be implemented in few hours.
Sustainability	This project can be made automated and has highest sustainability.
Social	This project is not expected to entail Ethical constraints.

REFERENCES

- [1]. Dimitrios Danopoulos, Christoforos Kachris, Dimitrios Soudris;“Acceleration of image classification with Caffe framework using FPGA”,2018 7th International Conference on Modern Circuits and Systems Technologies (MOCAST).
- [2]. Chen, Xiaofeng, Jingyu Ji, Yifan Zhang, Shaohui Mei,Manali Han, and Du, Q. (2018);”FPGA Based Implementation of Convolutional Neural Network for Hyperspectral Classification” IGARSS 2018 - 2018 IEEE International Geoscience and Remote Sensing Symposium.
- [3]. Shahmustafa Mujawar, Divya Kiran, Hariharan Ramasangu;“An Efficient CNN Architecture for Image, Classification on FPGA Accelerator”, 2018 Second International Conference on Advances in Electronics, Computer and Communications (ICAEECC-2018).
- [4]. <https://machinelearningmastery.com/how-to-develop-a-convolutional-neural-network-from-scratch-for-mnist-handwritten-digit-classification/>
- [5]. https://www.xilinx.com/support/documentation/university/Vivado-Teaching/HDL-Design/2013x/Nexys4/Verilog/docs-pdf/Vivado_tutorial.pdf
- [6]. <https://www.geeksforgeeks.org/image-classifier-using-cnn/>
- [7]. <https://www.edureka.co/blog/convolutional-neural-network/>
- [8]. <https://www.analyticsvidhya.com/blog/2018/12/guide-convolutional-neural-network-cnn/>
- [9]. <https://medium.com/@RaghavPrabhu/understanding-of-convolutional-neural-network-cnn-deep-learning-99760835f148>
- [10]. <http://www.fpgadeveloper.com/2014/08/creating-a-custom-ip-block-in-vivado.html>
- [11]. <https://reference.digilentinc.com/learn/programmable-logic/tutorials/zybo-creating-custom-ip-cores/start>

